

A1: Investigating Bias in Code Smell Detection Models

Student: Karthik Sridhar

Course: Data Science for Software Engineering

This analysis uses the 2 csv files which are 1. Data Class and 2. Feature Envy datasets from the official ESSeRE Lab `binary_class.zip` release. Each contains 420 examples from 74 projects. Their class balance describes these samples, not real-world smell prevalence.

1. Dataset setup

Going to upload these 2 csv files which was taken from the `binary_class.zip` file - `data-class.csv` and `feature-envy.csv` when prompted. Smelly is the positive label.

```
In [1]: from pathlib import Path

try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None
    IN_COLAB = False

required_csvs = ["data-class.csv", "feature-envy.csv"]
if IN_COLAB:
    missing_csvs = [name for name in required_csvs if not Path(name).is_file()]
    if missing_csvs:
        print("Upload these two files from the extracted binary_class folder:")
        print("  data-class.csv")
        print("  feature-envy.csv")
        colab_files.upload()
        missing_csvs = [name for name in required_csvs if not Path(name).is_file()]
        if missing_csvs:
            raise FileNotFoundError(
                "Upload both required CSV files. Still missing: " + ", ".join(mi
            )
        DATASET_DIR = Path(".")
        print("Colab upload ready:", ", ".join(required_csvs))
    else:
        DATASET_DIR = Path("data") / "binary_class"
        missing_csvs = [str(DATASET_DIR / name) for name in required_csvs if not (DA
        if missing_csvs:
            raise FileNotFoundError(
                "Local dataset files not found: " + ", ".join(missing_csvs)
            )
        print("Using local datasets in", DATASET_DIR)
```

Upload these two files from the extracted `binary_class` folder:

- `data-class.csv`
- `feature-envy.csv`

Choose Files No file chosen

Upload widget is only available when the cell has

been executed in the current browser session. Please rerun this cell to enable.

Saving data-class.csv to data-class.csv

Saving feature-envy.csv to feature-envy.csv

Colab upload ready: data-class.csv, feature-envy.csv

```
In [2]: %pip install -q pandas numpy matplotlib seaborn scikit-learn scipy openpyxl

import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.io import arff
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    ConfusionMatrixDisplay, balanced_accuracy_score, confusion_matrix,
    f1_score, precision_score, recall_score,
)
from sklearn.model_selection import StratifiedGroupKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

RANDOM_STATE = 42
POSITIVE_CLASS = 1
sns.set_theme(style="whitegrid", context="notebook")

DATASETS = {
    "Data Class": {
        "path": str(DATASET_DIR / "data-class.csv"),
        "target_column": "is_data_class",
        "positive_label": True,
        "negative_label": False,
        "drop_columns": ["IDType", "project", "package", "complextypes"],
    },
    "Feature Envy": {
        "path": str(DATASET_DIR / "feature-envy.csv"),
        "target_column": "is_feature_envy",
        "positive_label": True,
        "negative_label": False,
        "drop_columns": ["IDMethod", "project", "package", "complextypes", "method"],
    },
}

TEST_SIZE = 0.20
CV_FOLDS = 5
TOP_METRICS_TO_PLOT = 6
CORRELATION_THRESHOLD = 0.85

print("Configured Data Class and Feature Envy datasets. Smelly instances are the
```

Configured Data Class and Feature Envy datasets. Smelly instances are the positive class.

```
In [3]: def read_dataset(path):
        path = Path(path).expanduser()
```

```

if not path.is_file():
    raise FileNotFoundError(f"Dataset file not found: {path.resolve()}")
suffix = path.suffix.lower()
if suffix == ".csv":
    try:
        return pd.read_csv(path)
    except pd.errors.ParserError:
        return pd.read_csv(path, quotechar="")
if suffix in {".xlsx", ".xls"}:
    return pd.read_excel(path)
if suffix == ".arff":
    records, _ = arff.loadarff(path)
    frame = pd.DataFrame(records)
    for column in frame.select_dtypes(include=["object"]).columns:
        frame[column] = frame[column].map(
            lambda value: value.decode("utf-8") if isinstance(value, bytes)
        )
    return frame
raise ValueError(f"Unsupported file type {suffix!r}; use CSV, Excel, or ARFF")

def prepare_dataset(name, config):
    required = ("path", "target_column", "positive_label", "negative_label")
    missing_config = [
        key for key in required
        if config.get(key) is None or config.get(key) == ""
    ]
    if missing_config:
        raise ValueError(
            f"Configure {name}: missing {' '.join(missing_config)} in DATASETS."
        )
    frame = read_dataset(config["path"]).copy()
    target = config["target_column"]
    if target not in frame.columns:
        raise KeyError(f"{name}: target {target!r} not found. Columns: {frame.co
    frame = frame.loc[frame[target].notna()].copy()
    groups = frame["project"].copy() if "project" in frame.columns else pd.Series
    frame = frame.drop(columns=[c for c in config.get("drop_columns", []) if c i
    observed = set(frame[target].unique())
    expected = {config["positive_label"], config["negative_label"]}
    if observed != expected:
        raise ValueError(
            f"{name}: labels found {sorted(map(str, observed))}; expected exactl
            f"{sorted(map(str, expected))}. Check values and filter unrelated la
        )
    y = frame.pop(target).map({config["negative_label"]: 0, config["positive_lab
    X = frame.apply(pd.to_numeric, errors="coerce")
    all_missing = X.columns[X.isna().all()].tolist()
    if all_missing:
        warnings.warn(f"{name}: dropping non-numeric/all-missing columns: {all_m
        X = X.drop(columns=all_missing)
    if X.empty:
        raise ValueError(f"{name}: no numeric feature columns remain. Check file
    constant = X.columns[X.nunique(dropna=True) <= 1].tolist()
    near_constant = []
    for column in X.columns.difference(constant):
        frequencies = X[column].value_counts(normalize=True, dropna=True)
        if not frequencies.empty and frequencies.iloc[0] >= 0.99:
            near_constant.append(column)
    return {
        "raw": frame, "X": X, "y": y, "groups": groups,

```

```

        "constant": constant, "near_constant": near_constant, "target": target,
    }

data = {name: prepare_dataset(name, config) for name, config in DATASETS.items()}
print("Loaded datasets:", {name: item['X'].shape for name, item in data.items()})
print("Project groups:", {name: item['groups'].nunique() for name, item in data.items()})

```

Loaded datasets: {'Data Class': (420, 61), 'Feature Envy': (420, 82)}
 Project groups: {'Data Class': 74, 'Feature Envy': 74}

2. Dataset overview and class distribution

The table reports dataset size, class counts, missing values, duplicates, and constant or near-constant features. The plots show the class balance in each sample.

```

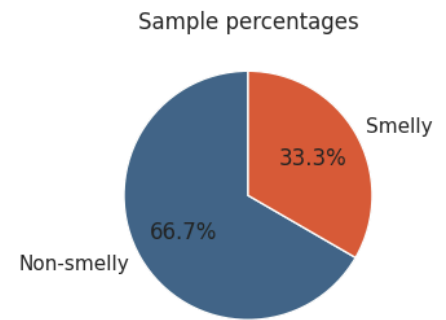
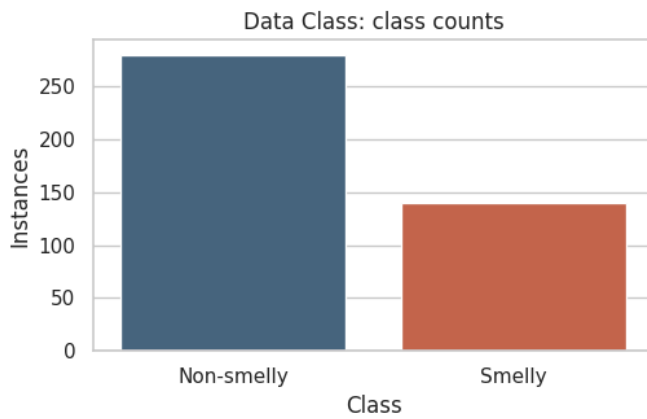
In [4]: overview_rows = []
for name, item in data.items():
    X, y = item["X"], item["y"]
    raw_with_target = pd.concat([X, y.rename(item["target"])], axis=1)
    counts = y.value_counts().reindex([0, 1], fill_value=0)
    percentages = y.value_counts(normalize=True).reindex([0, 1], fill_value=0) * 100
    overview_rows.append({
        "Dataset": name, "Rows": len(X), "Numeric features": X.shape[1],
        "Smelly (n, %)": f"{counts[1]} ({percentages[1]:.1f}%)",
        "Non-smelly (n, %)": f"{counts[0]} ({percentages[0]:.1f}%)",
        "Missing feature cells": int(X.isna().sum().sum()),
        "Exact duplicate metric/label rows": int(raw_with_target.duplicated().sum()),
        "Constant features detected": len(item["constant"]),
        "Near-constant features (>=99% one value)": len(item["near_constant"]),
    })
    print(f"\n{name}: target={item['target']}!r; constant features={item['constant']}!r; near-constant features={item['near_constant']}!r")
    display(pd.DataFrame({"count": counts, "percent": percentages.round(2)}).rename(columns={"count": "count", "percent": "percent"}))
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
    labels = y.map({0: "Non-smelly", 1: "Smelly"})
    sns.countplot(
        x=labels, order=["Non-smelly", "Smelly"], hue=labels, legend=False,
        ax=axes[0], palette={"Non-smelly": "#416788", "Smelly": "#d95d39"},
    )
    axes[0].set(title=f"{name}: class counts", xlabel="Class", ylabel="Instances")
    axes[1].pie(counts.values, labels=["Non-smelly", "Smelly"], autopct="%0.1f%%")
    axes[1].set_title("Sample percentages")
    plt.tight_layout(); plt.show()

overview = pd.DataFrame(overview_rows).set_index("Dataset")
display(overview)

```

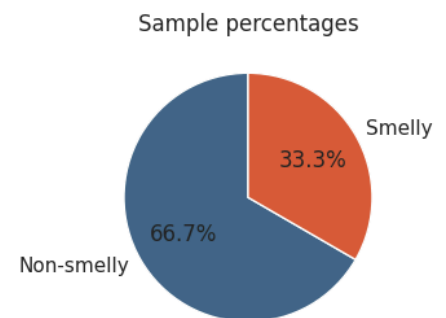
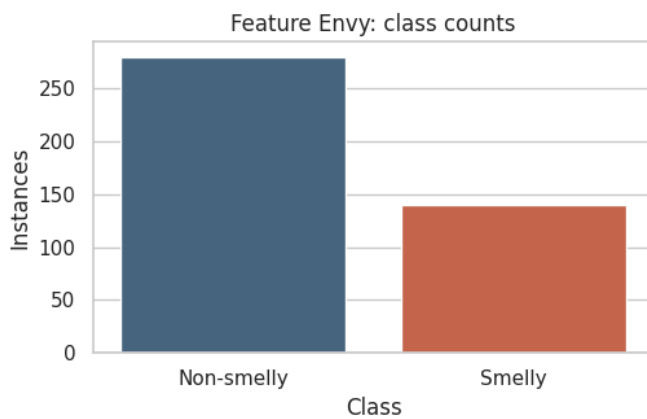
Data Class: target='is_data_class'; constant features=[]
 Near-constant features=[]

	count	percent
is_data_class		
Non-smelly	280	66.67
Smelly	140	33.33



Feature Envy: target='is_feature_envy'; constant features=[]
Near-constant features=['isStatic_type']

	count	percent
is_feature_envy		
Non-smelly	280	66.67
Smelly	140	33.33



	Rows	Numeric features	Smelly (n, %)	Non-smelly (n, %)	Missing feature cells	Exact duplicate metric/label rows	Constant features detected	Near-constant features (>=99% one value)
Dataset								
Data Class	420	61	140 (33.3%)	280 (66.7%)	75	2	0	0
Feature Envy	420	82	140 (33.3%)	280 (66.7%)	92	13	0	1

3. Software metric distributions

These plots compare metric values for smelly and non-smelly examples. The summaries report central values, spread, skew, outliers, and overlap.

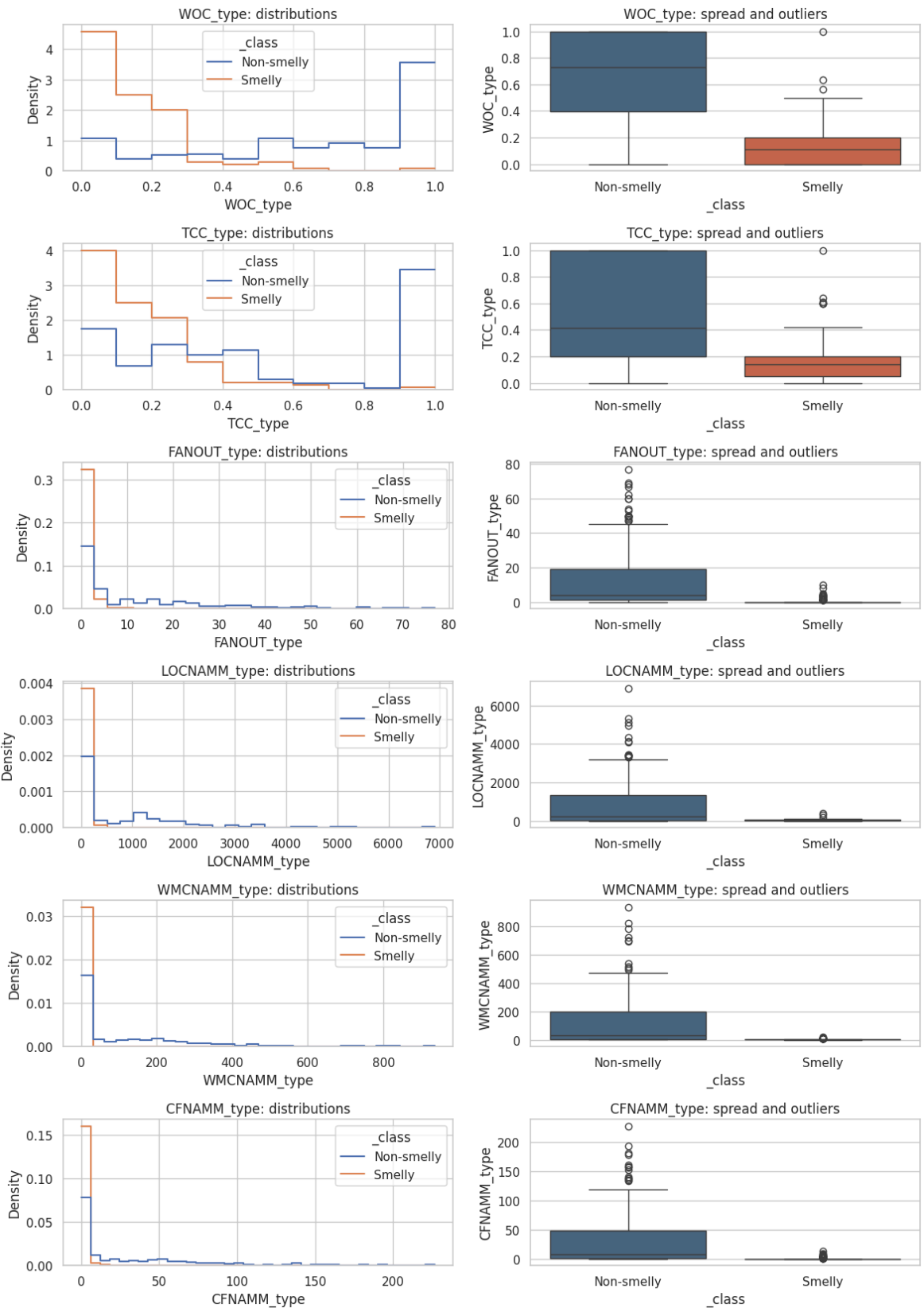
```
In [5]: distribution_summaries = {}
for name, item in data.items():
    X, y = item["X"], item["y"]
    summary = X.assign(_class=y.map({0: "Non-smelly", 1: "Smelly"})).groupby("_class")
    distribution_summaries[name] = summary
    effect = (X[y.eq(1)].mean() - X[y.eq(0)].mean()) / X.std(ddof=1).replace(0, 1)
    selected = effect.abs().sort_values(ascending=False).head(min(TOP_METRICS_TO_DISPLAY, 10))
    print(f"\n{name}: class-wise metric summary (count, mean, median, std, min, max, skew, kurtosis, ...):")
    display(summary.loc[:, pd.IndexSlice[selected, :]])
    print("Largest absolute standardized mean differences (descriptive):", {m: r for m, r in selected.items()})
    plot_frame = X[selected].assign(_class=y.map({0: "Non-smelly", 1: "Smelly"}))
    fig, axes = plt.subplots(len(selected), 2, figsize=(12, max(3.0, 2.8 * len(selected))))
    for row, metric in enumerate(selected):
        sns.histplot(data=plot_frame, x=metric, hue="_class", stat="density", color_palette="magma", ax=axes[row, 0])
        axes[row, 0].set_title(f"{metric}: distributions")
        sns.boxplot(data=plot_frame, x="_class", y=metric, order=["Non-smelly", "Smelly"], ax=axes[row, 1])
        axes[row, 1].set_title(f"{metric}: spread and outliers")
    plt.tight_layout(); plt.show()
    for metric in selected:
        clean = X.loc[y.eq(0), metric].dropna()
        smelly = X.loc[y.eq(1), metric].dropna()
        clean_iqr = clean.quantile([0.25, 0.75])
        smelly_iqr = smelly.quantile([0.25, 0.75])
        overlap = max(clean_iqr.iloc[0], smelly_iqr.iloc[0]) <= min(clean_iqr.iloc[1], smelly_iqr.iloc[1])
        central_ranges = "overlap" if overlap else "do not overlap"
        print(
            f"{metric}: median clean={clean.median():.3g}, smelly={smelly.median():.3g}; "
            f"skew clean={clean.skew():.2f}, smelly={smelly.skew():.2f}; "
            f"IQRs {central_ranges}; observed ranges clean=[{clean.min():.3g}, {clean.max():.3g}], "
            f"smelly=[{smelly.min():.3g}, {smelly.max():.3g}]."
        )
```

Data Class: class-wise metric summary (count, mean, median, std, min, max, skew)

	WOC_type							TCC_type				WMC_type	
	count	mean	median	std	min	max	skew	count	mean	median	...	min	max
_class													
Non-smelly	252	0.652	0.733	0.341	0.0	1.0	-0.604	280	0.518	0.414	...	1	1
Smelly	140	0.135	0.111	0.154	0.0	1.0	2.027	140	0.156	0.143	...	0	0

2 rows × 42 columns

Largest absolute standardized mean differences (descriptive): {'WOC_type': -1.36, 'TCC_type': -0.98, 'FANOUT_type': -0.84, 'LOCNAMM_type': -0.81, 'WMCNAMM_type': -0.81, 'CFNAMM_type': -0.79}



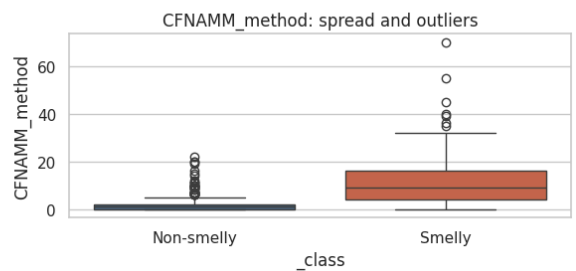
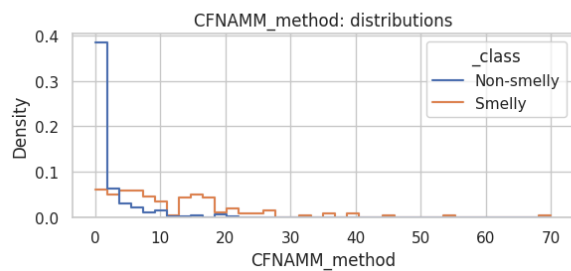
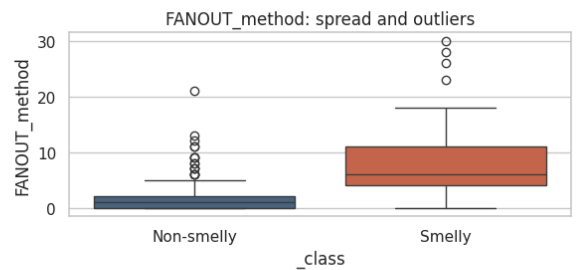
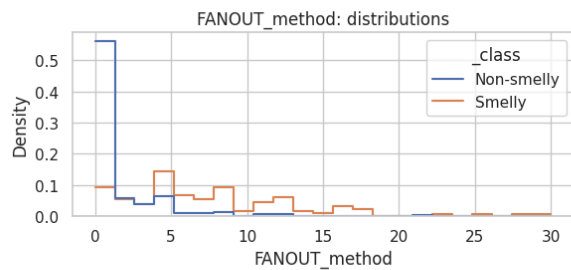
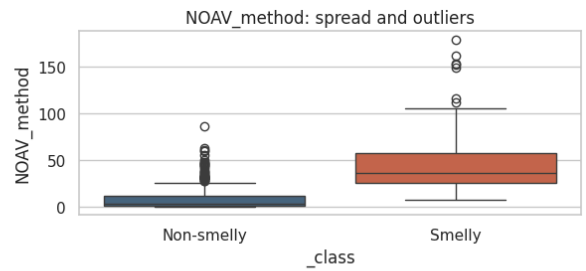
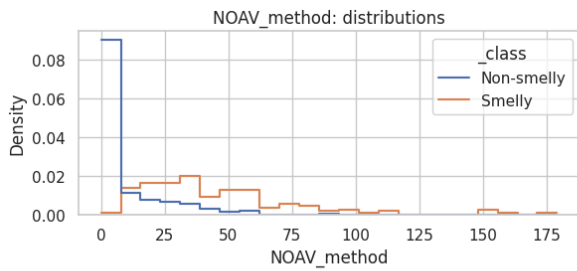
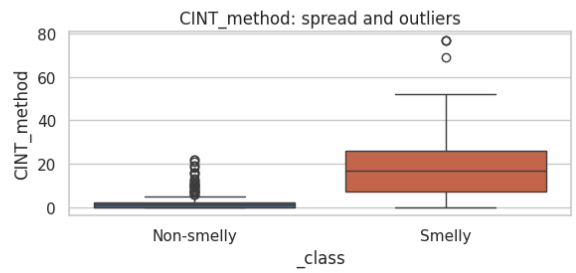
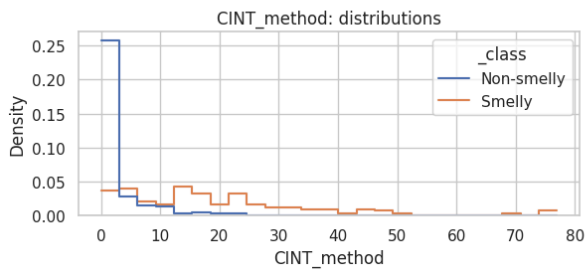
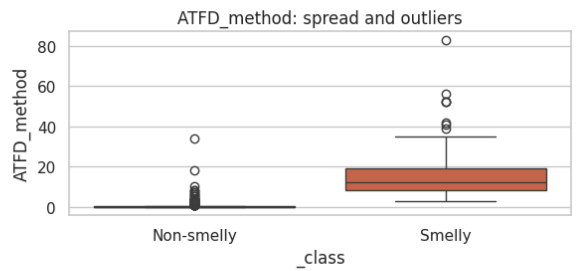
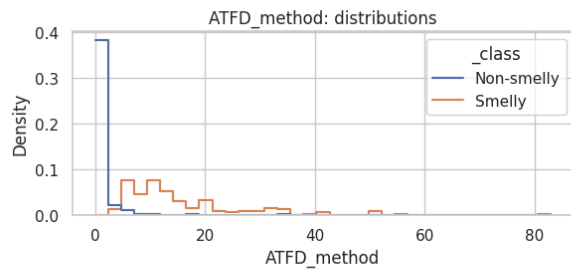
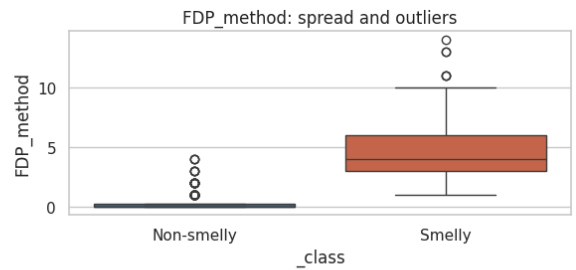
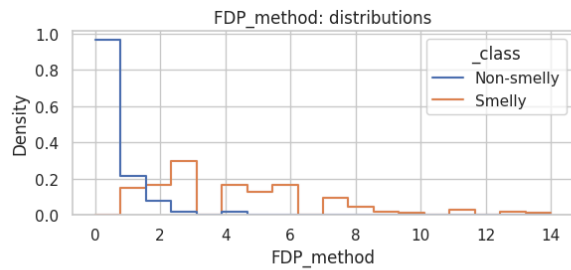
WOC_type: median clean=0.733, smelly=0.111; skew clean=-0.60, smelly=2.03; IQRs do not overlap; observed ranges clean=[0, 1], smelly=[0, 1].
TCC_type: median clean=0.414, smelly=0.143; skew clean=0.16, smelly=2.08; IQRs overlap; observed ranges clean=[0, 1], smelly=[0, 1].
FANOUT_type: median clean=4, smelly=0; skew clean=1.66, smelly=4.04; IQRs do not overlap; observed ranges clean=[0, 77], smelly=[0, 10].
LOCNAMM_type: median clean=248, smelly=29.5; skew clean=2.03, smelly=4.03; IQRs overlap; observed ranges clean=[3, 6.91e+03], smelly=[0, 380].
WMCNAMM_type: median clean=29.5, smelly=1; skew clean=1.96, smelly=2.38; IQRs do not overlap; observed ranges clean=[1, 937], smelly=[0, 18].
CFNAMM_type: median clean=7, smelly=0; skew clean=1.91, smelly=4.01; IQRs do not overlap; observed ranges clean=[0, 227], smelly=[0, 13].

Feature Envy: class-wise metric summary (count, mean, median, std, min, max, skew)

	FDP_method							ATFD_method ... FANOUT				
	count	mean	median	std	min	max	skew	count	mean	median	...	min
_class												
Non-smelly	280	0.371	0.0	0.756	0	4	2.425	280	0.814	0.0	...	0
Smelly	140	4.350	4.0	2.658	1	14	1.188	140	15.964	12.0	...	0

2 rows × 42 columns

◀  ▶
Largest absolute standardized mean differences (descriptive): {'FDP_method': 1.59, 'ATFD_method': 1.47, 'CINT_method': 1.36, 'NOAV_method': 1.32, 'FANOUT_method': 1.27, 'CFNAMM_method': 1.19}



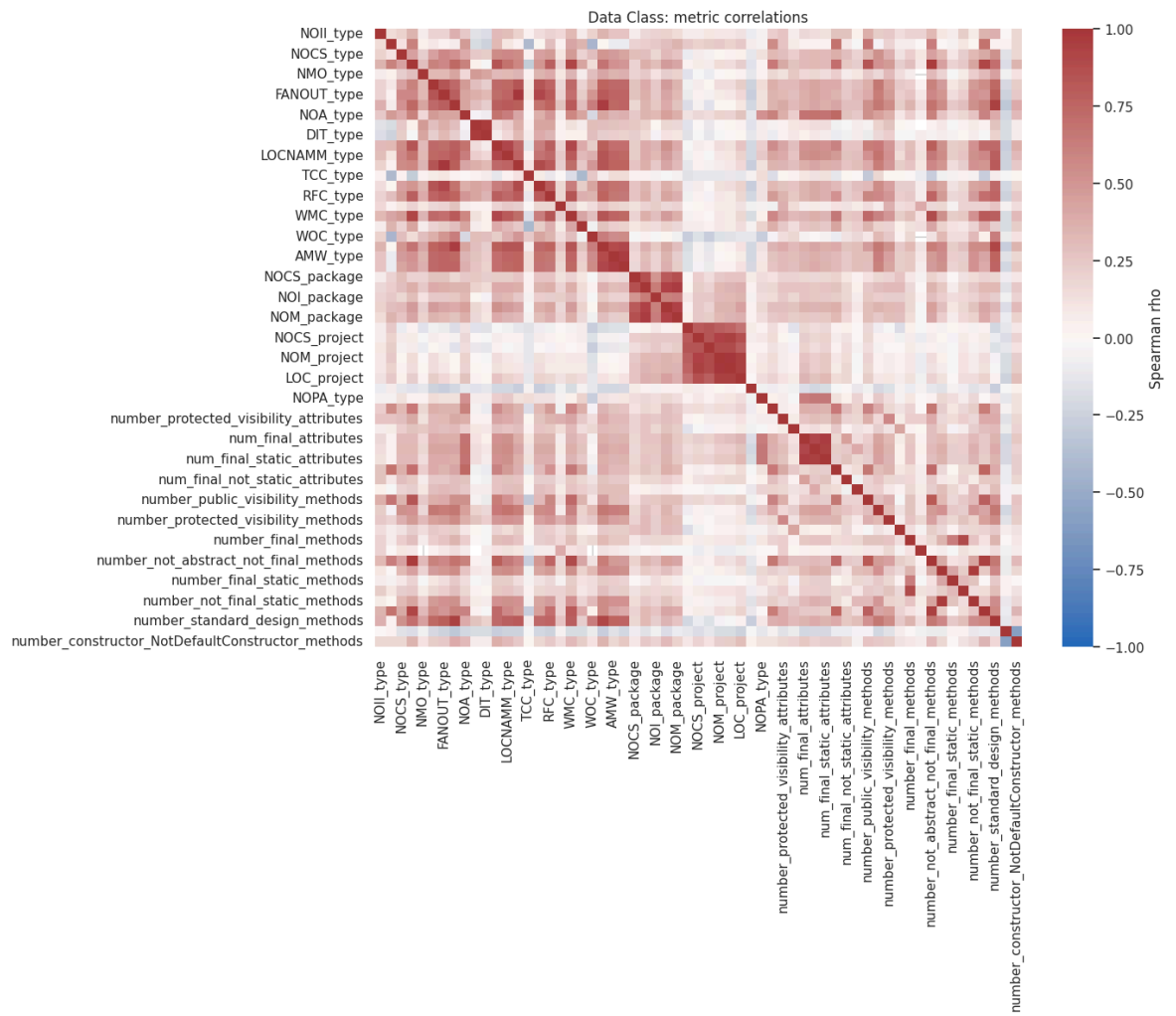
FDP_method: median clean=0, smelly=4; skew clean=2.43, smelly=1.19; IQRs do not overlap; observed ranges clean=[0, 4], smelly=[1, 14].
 ATFD_method: median clean=0, smelly=12; skew clean=8.11, smelly=2.26; IQRs do not overlap; observed ranges clean=[0, 34], smelly=[3, 83].
 CINT_method: median clean=1, smelly=17; skew clean=2.64, smelly=1.48; IQRs do not overlap; observed ranges clean=[0, 22], smelly=[0, 77].
 NOAV_method: median clean=3, smelly=35.5; skew clean=2.17, smelly=1.73; IQRs do not overlap; observed ranges clean=[0, 86], smelly=[7, 179].
 FANOUT_method: median clean=1, smelly=6; skew clean=3.36, smelly=1.30; IQRs do not overlap; observed ranges clean=[0, 21], smelly=[0, 30].
 CFNAMM_method: median clean=1, smelly=9; skew clean=2.90, smelly=2.06; IQRs do not overlap; observed ranges clean=[0, 22], smelly=[0, 70].

4. Correlation analysis

Spearman correlations identify metrics that may capture similar information. Correlation alone does not establish bias.

```
In [6]: correlations = {}
for name, item in data.items():
    corr = item["X"].corr(method="spearman")
    correlations[name] = corr
    upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
    pairs = (upper.stack().rename("Spearman rho").to_frame()
             .assign(abs_rho=lambda table: table["Spearman rho"].abs())
             .query("abs_rho >= @CORRELATION_THRESHOLD")
             .sort_values("abs_rho", ascending=False))
    print(f"\n{name}: feature correlation matrix (Spearman)")
    fig, ax = plt.subplots(figsize=(min(14, max(7, 0.55 * len(corr.columns))),
    sns.heatmap(corr, cmap="vlag", center=0, vmin=-1, vmax=1, ax=ax, cbar_kws={"
    ax.set_title(f"{name}: metric correlations"); plt.tight_layout(); plt.show()
    print(f"Pairs with |rho| >= {CORRELATION_THRESHOLD}:")
    display(pairs if not pairs.empty else pd.DataFrame(columns=["Spearman rho",
```

Data Class: feature correlation matrix (Spearman)

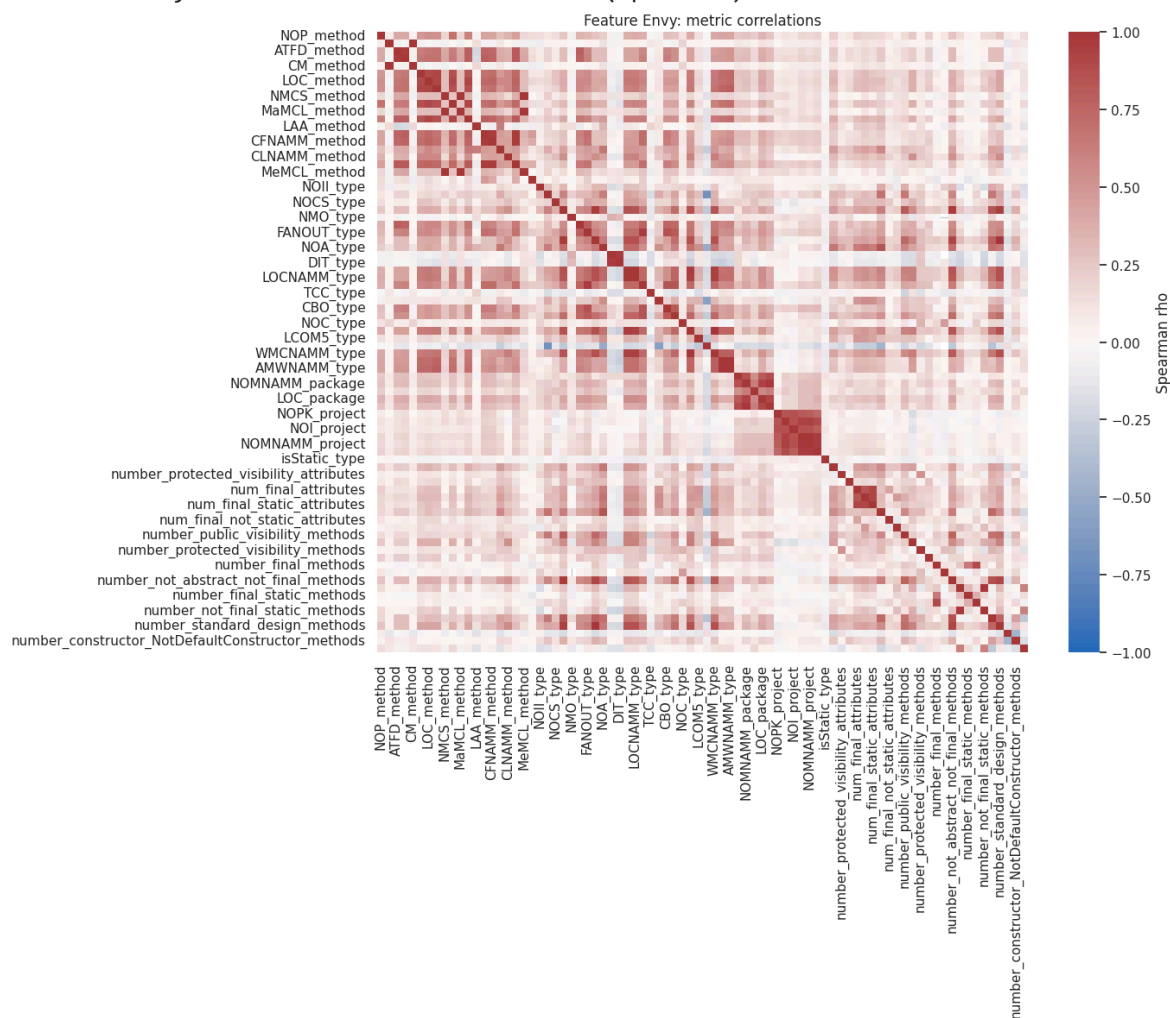


Pairs with $|\rho| \geq 0.85$:

			Spearman rho
	NOM_project	NOMNAMM_project	0.992047
	FANOUT_type	CFNAMM_type	0.984548
	NOM_type	number_not_abstract_not_final_methods	0.973909
	number_static_methods	number_not_final_static_methods	0.971779
	NIM_type	DIT_type	0.971690
	NOMNAMM_package	NOM_package	0.959118
	AMW_type	AMWNAMM_type	0.958044
	num_final_attributes	num_final_static_attributes	0.957508
	NOMNAMM_type	WMCNAMM_type	0.956822
	NOM_project	LOC_project	0.950154
	num_static_attributes	num_final_static_attributes	0.946440
	number_not_abstract_not_final_methods	number_not_final_not_static_methods	0.943278
	NOMNAMM_type	number_standard_design_methods	0.942775
	NOMNAMM_project	LOC_project	0.942764
	NOCS_project	NOMNAMM_project	0.940085
	NOMNAMM_package	LOC_package	0.934012
	NOCS_project	NOM_project	0.931556
	LOC_package	NOM_package	0.930185
	NOM_type	WMC_type	0.920678
	WMCNAMM_type	number_standard_design_methods	0.920414
	NOM_type	number_not_final_not_static_methods	0.919671
	LOC_type	LOCNAMM_type	0.904877
	FANOUT_type	CBO_type	0.902742
	LOC_type	WMC_type	0.900364
	num_final_attributes	num_static_attributes	0.900038
	WMCNAMM_type	AMWNAMM_type	0.900021
	CFNAMM_type	CBO_type	0.897263
	NOI_project	NOMNAMM_project	0.889311
	number_final_methods	number_final_not_static_methods	0.888642
	NOI_project	NOM_project	0.888506
	WMC_type	number_not_abstract_not_final_methods	0.887074
	WMCNAMM_type	AMW_type	0.876654

		Spearman rho
NOCS_package	NOM_package	0.873710
	NOMNAMM_package	0.870395
NOCS_project	LOC_project	0.853398

Feature Envy: feature correlation matrix (Spearman)



Pairs with $|\rho| \geq 0.85$:

		Spearman rho	
	MaMCL_method	MeMCL_method	0.999666 C
	NMCS_method	MaMCL_method	0.998877 C
		MeMCL_method	0.998572 C
	LOC_type	LOCNAMM_type	0.996681 C
	CC_method	CM_method	0.995716 C
	...	...	...
	NOCS_project	NOI_project	0.856568 C
	FANOUT_type	CBO_type	0.855855 C
number_not_abstract_not_final_methods	number_not_final_not_static_methods		0.855248 C
	NOPK_project	NOI_project	0.854339 C
		NOCS_project	0.851524 C

64 rows × 2 columns



5. Model training and evaluation

A Decision Tree and a class-weighted Random Forest are compared using training-only grouped cross-validation. Entire projects are held out for testing. Imputation is fitted inside each pipeline. Smelly is the positive class; results include precision, recall, F1, balanced accuracy, and confusion matrices.

```
In [7]: SCORING = {
    "precision": "precision", "recall": "recall",
    "f1": "f1", "balanced_accuracy": "balanced_accuracy",
}
MODEL_FACTORIES = {
    "Decision Tree": lambda: DecisionTreeClassifier(class_weight="balanced", ran
    "Random Forest": lambda: RandomForestClassifier(n_estimators=300, class_weig
}

def make_pipeline(features):
    numeric = Pipeline([("imputer", SimpleImputer(strategy="median"))])
    prep = ColumnTransformer([("numeric", numeric, features)], remainder="drop")
    return Pipeline([("preprocess", prep), ("classifier", None)])

def build_model(features, factory):
    pipeline = make_pipeline(features)
    pipeline.set_params(classifier=factory())
    return pipeline

cv_results = []
test_results = []
test_predictions = {}
splits = {}
```

```

for name, item in data.items():
    X, y, groups = item["X"], item["y"], item["groups"]
    labeled_metrics = X.assign(_target=y)
    unique_rows = ~labeled_metrics.duplicated()
    removed_duplicates = int((~unique_rows).sum())
    X, y, groups = X.loc[unique_rows], y.loc[unique_rows], groups.loc[unique_row
    print(f"\n{name}: excluding {removed_duplicates} exact metric/label duplicat

    outer_cv = StratifiedGroupKFold(n_splits=round(1 / TEST_SIZE), shuffle=True,
    train_indices, test_indices = next(outer_cv.split(X, y, groups))
    X_train, X_test = X.iloc[train_indices], X.iloc[test_indices]
    y_train, y_test = y.iloc[train_indices], y.iloc[test_indices]
    groups_train, groups_test = groups.iloc[train_indices], groups.iloc[test_ind
    if set(groups_train).intersection(groups_test):
        raise RuntimeError(f"{name}: project groups leaked across train and test
    print(f"Train: {len(X_train)} rows / {groups_train.nunique()} projects; test
    print("Test class counts:", y_test.value_counts().reindex([0, 1], fill_value
    splits[name] = (X_train, X_test, y_train, y_test, groups_train, groups_test)
    cv = StratifiedGroupKFold(n_splits=min(CV_FOLDS, groups_train.nunique()), sh

for model_name, factory in MODEL_FACTORIES.items():
    features = X_train.columns.tolist()
    model = build_model(features, factory)
    scores = cross_validate(
        model, X_train, y_train, groups=groups_train, cv=cv,
        scoring=SCORING, n_jobs=1, error_score="raise",
    )
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    test_predictions[(name, model_name)] = prediction
    cv_results.append({
        "Dataset": name, "Model": model_name,
        "CV precision mean": scores["test_precision"].mean(),
        "CV recall mean": scores["test_recall"].mean(),
        "CV F1 mean": scores["test_f1"].mean(),
        "CV F1 std": scores["test_f1"].std(),
        "CV balanced accuracy mean": scores["test_balanced_accuracy"].mean()
    })
    result = {
        "Dataset": name, "Model": model_name,
        "Precision": precision_score(y_test, prediction, zero_division=0),
        "Recall": recall_score(y_test, prediction, zero_division=0),
        "F1": f1_score(y_test, prediction, zero_division=0),
        "Balanced accuracy": balanced_accuracy_score(y_test, prediction),
    }
    test_results.append(result)
    print(f"{model_name} held-out project metrics:", {key: round(value, 3) f
    fig, ax = plt.subplots(figsize=(4.5, 4))
    ConfusionMatrixDisplay.from_predictions(
        y_test, prediction, display_labels=["Non-smelly", "Smelly"],
        cmap="Blues", colorbar=False, ax=ax,
    )
    ax.set_title(f"{name}: {model_name} (project-held-out)")
    plt.tight_layout(); plt.show()

print("Training-only project-grouped CV results:")
display(pd.DataFrame(cv_results).round(3))
print("Held-out project test results (positive class: smelly):")
display(pd.DataFrame(test_results).round(3))

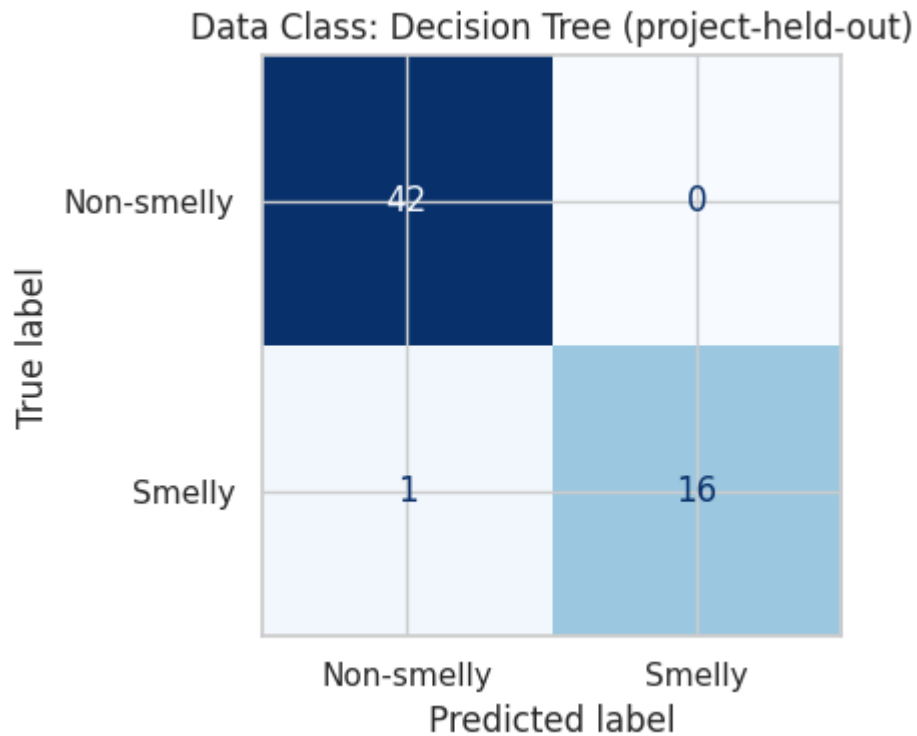
```

Data Class: excluding 2 exact metric/label duplicates from model evaluation

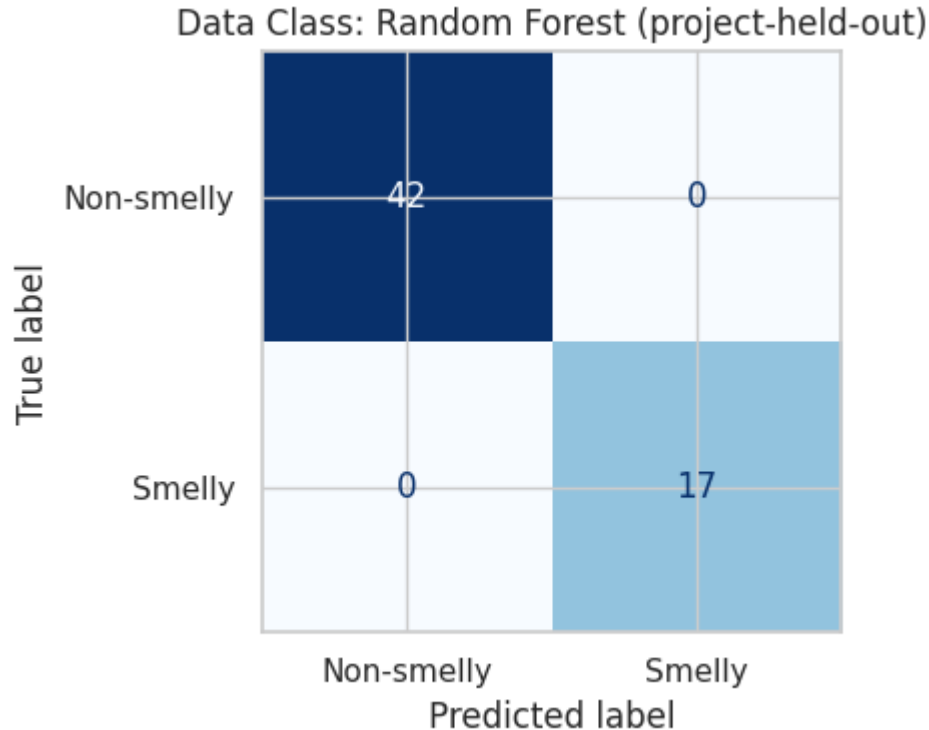
Train: 359 rows / 61 projects; test: 59 rows / 13 projects

Test class counts: {'Non-smelly': 42, 'Smelly': 17}

Decision Tree held-out project metrics: {'Precision': 1.0, 'Recall': 0.941, 'F1': 0.97, 'Balanced accuracy': np.float64(0.971)}



Random Forest held-out project metrics: {'Precision': 1.0, 'Recall': 1.0, 'F1': 1.0, 'Balanced accuracy': np.float64(1.0)}



Feature Envy: excluding 13 exact metric/label duplicates from model evaluation

Train: 308 rows / 57 projects; test: 99 rows / 17 projects

Test class counts: {'Non-smelly': 64, 'Smelly': 35}

Decision Tree held-out project metrics: {'Precision': 0.971, 'Recall': 0.971, 'F1': 0.971, 'Balanced accuracy': np.float64(0.978)}

Feature Envy: Decision Tree (project-held-out)

True label	Non-smelly	63	1
	Smelly	1	34
		Non-smelly	Smelly
		Predicted label	

Random Forest held-out project metrics: {'Precision': 1.0, 'Recall': 0.971, 'F1': 0.986, 'Balanced accuracy': np.float64(0.986)}

Feature Envy: Random Forest (project-held-out)

True label	Non-smelly	64	0
	Smelly	1	34
		Non-smelly	Smelly
		Predicted label	

Training-only project-grouped CV results:

	Dataset	Model	CV precision mean	CV recall mean	CV F1 mean	CV F1 std	CV balanced accuracy mean
0	Data Class	Decision Tree	0.952	0.954	0.951	0.020	0.963
1	Data Class	Random Forest	0.971	0.953	0.960	0.040	0.968
2	Feature Envy	Decision Tree	0.888	0.899	0.891	0.043	0.922
3	Feature Envy	Random Forest	0.923	0.947	0.934	0.029	0.953

Held-out project test results (positive class: smelly):

	Dataset	Model	Precision	Recall	F1	Balanced accuracy
0	Data Class	Decision Tree	1.000	0.941	0.970	0.971
1	Data Class	Random Forest	1.000	1.000	1.000	1.000
2	Feature Envy	Decision Tree	0.971	0.971	0.971	0.978
3	Feature Envy	Random Forest	1.000	0.971	0.986	0.986

6. Size-metric comparison

The models are also evaluated after removing selected size-related metrics. This checks whether performance changes when those metrics are unavailable.

```
In [8]: SIZE_PATTERN = re.compile(r"^(^|_)(LOC|WMC|NOM|NOA|NOMNAMM|AMW|NPM|NPA|TLOC|LLOC)
ablation_rows = []
for name, item in data.items():
    X_train, X_test, y_train, y_test, groups_train, groups_test = splits[name]
    size_features = [column for column in X_train.columns if SIZE_PATTERN.search(column)]
    reduced_features = [column for column in X_train.columns if column not in size_features]
    print(f"\n{name}: size-related columns removed: {size_features}")
    if not size_features or not reduced_features:
        print("Ablation skipped because no suitable size-feature subset was found")
        continue
    cv = StratifiedGroupKFold(n_splits=min(CV_FOLDS, groups_train.nunique()), shuffle=True)
    for model_name, factory in MODEL_FACTORIES.items():
        for setting, features in [("All metrics", X_train.columns.tolist()), ("Without size metrics", reduced_features)]:
            model = build_model(features, factory)
            scores = cross_validate(
                model, X_train[features], y_train, groups=groups_train, cv=cv,
                scoring=SCORING, n_jobs=1, error_score="raise",
            )
            model.fit(X_train[features], y_train)
            prediction = model.predict(X_test[features])
            ablation_rows.append({
                "Dataset": name, "Model": model_name, "Features": setting,
                "CV F1 mean": scores["test_f1"].mean(),
                "CV balanced accuracy mean": scores["test_balanced_accuracy"].mean(),
                "Test F1": f1_score(y_test, prediction, zero_division=0),
                "Test balanced accuracy": balanced_accuracy_score(y_test, prediction)
```

```

    })
    ablation = pd.DataFrame(ablation_rows)
    print("Grouped-CV and project-held-out size-feature ablation:")
    display(ablation.round(3))

```

Data Class: size-related columns removed: ['NOM_type', 'NOMNAMM_type', 'NOA_type', 'LOC_type', 'WMC_type', 'AMW_type', 'NOMNAMM_package', 'LOC_package', 'NOM_package', 'NOM_project', 'NOMNAMM_project', 'LOC_project']

Feature Envy: size-related columns removed: ['LOC_method', 'NOM_type', 'NOMNAMM_type', 'NOA_type', 'LOC_type', 'WMC_type', 'AMW_type', 'NOMNAMM_package', 'LOC_package', 'NOM_package', 'NOM_project', 'NOMNAMM_project', 'LOC_project']

Grouped-CV and project-held-out size-feature ablation:

	Dataset	Model	Features	CV F1 mean	CV balanced accuracy mean	Test F1	Test balanced accuracy
0	Data Class	Decision Tree	All metrics	0.951	0.963	0.970	0.971
1	Data Class	Decision Tree	Without size metrics	0.953	0.962	0.970	0.971
2	Data Class	Random Forest	All metrics	0.960	0.968	1.000	1.000
3	Data Class	Random Forest	Without size metrics	0.970	0.977	1.000	1.000
4	Feature Envy	Decision Tree	All metrics	0.891	0.922	0.971	0.978
5	Feature Envy	Decision Tree	Without size metrics	0.894	0.920	0.957	0.964
6	Feature Envy	Random Forest	All metrics	0.934	0.953	0.986	0.986
7	Feature Envy	Random Forest	Without size metrics	0.923	0.942	0.986	0.986

7. Findings and limitations

Both samples contain 140 smelly and 280 non-smelly examples. Data Class has 75 missing values and 2 duplicate metric/label rows; Feature Envy has 92 missing values, 13 duplicate rows, and one near-constant feature. Missing values are imputed during model training.

Metrics differ by label: Data Class WOC medians are 0.733 (non-smelly) and 0.111 (smelly); Feature Envy FDP medians are 0 and 4. Several metric pairs are strongly correlated, including `NOM_project` / `NOMNAMM_project` (rho 0.992) and `MaMCL_method` / `MeMCL_method` (rho 0.9997). These patterns may make the labels easier to predict from this sample's metrics. Removing selected size metrics changed Random Forest F1 only slightly: grouped-CV F1 was 0.962 to 0.976 for Data Class and 0.939 to 0.933 for Feature Envy.

The Random Forest scored F1 1.000 on the held-out projects in both datasets. However, the samples have a selected 1:2 class balance and come from one archive and labeling process. These results do not establish real-world prevalence or guarantee performance on other projects or labeling methods.